Counting Unlabeled Simple Graphs via Group Actions: A Detailed Burnside–Pólya Derivation - CS215 Discrete Math (H) Project

Abstract

We present a self-contained group-theoretic derivation of the classical enumeration formula for the number of non-isomorphic (unlabeled) simple undirected graphs on n vertices. The counting problem is framed as an orbit-counting task under the natural action of the symmetric group S_n on the set of labeled graphs. We develop the necessary algebraic background in full detail: group actions, orbits, stabilizers, the orbit–stabilizer theorem, and Burnside's lemma. We then specialize Pólya-style reasoning to edge-colorings of the complete graph with two colors (edge present/absent), reducing the computation of fixed objects to counting cycles of the induced permutation on the set of unordered vertex pairs. A central contribution is a complete derivation of the fixed-point exponent

$$c(\sigma) = \sum_{i=1}^{r} \left\lfloor \frac{b_i}{2} \right\rfloor + \sum_{1 \le i < j \le r} \gcd(b_i, b_j),$$

where $\sigma \in S_n$ has cycle lengths $b_1, \ldots, b_r$. Finally, we regroup the Burnside sum by cycle type and derive the standard closed formula involving integer partitions:

$$a(n) = \sum_{\lambda \vdash n} \frac{2^{c(\lambda)}}{\prod_{\ell \ge 1} \ell^{m_\ell} m_\ell!},$$

where λ has m_ℓ cycles of length ℓ and $c(\lambda)$ is computed from these multiplicities. We conclude with algorithmic remarks and references to classical sources in enumeration theory.

1. Introduction: unlabeled graphs as orbit classes

Let $[n] = \{1, 2, \dots, n\}$. A labeled simple undirected graph on $[n]$ is determined by its edge set

$$E \subseteq \binom{[n]}{2} = \{\{i, j\} : 1 \le i < j \le n\}.$$

Hence the set X_n of all labeled graphs on $[n]$ has cardinality

$$|X_n| = 2^{\binom{n}{2}}.$$

Two labeled graphs are isomorphic if one can be obtained from the other by relabeling vertices. This suggests the following standard abstraction:

- The relabelings form the symmetric group S_n.
- The set of unlabeled graphs is the set of orbits of X_n under the action of S_n.

Therefore, the desired quantity is the orbit count

$$a(n) = |X_n / S_n|.$$

The remainder of this essay develops the necessary group theory and executes the orbit count explicitly.

---

2. Algebraic preliminaries

2.1. Groups and permutations

A group is a pair $(G, \cdot)$ where G is a set and $\cdot$ is a binary operation on G satisfying:

1. (Closure) for all $a, b \in G$, $a \cdot b \in G$.
2. (Associativity) for all $a, b, c \in G$, $(a \cdot b) \cdot c = a \cdot (b \cdot c)$.
3. (Identity) there exists $e \in G$ such that for all $a \in G$, $e \cdot a = a \cdot e = a$.
4. (Inverses) for each $a \in G$ there exists $a^{-1} \in G$ such that $a \cdot a^{-1} = a^{-1} \cdot a = e$.

The symmetric group S_n is the group of all bijections $\sigma : [n] \to [n]$ under composition. Each $\sigma \in S_n$ admits a cycle decomposition into disjoint cycles; the multiset of cycle lengths is called its cycle type.

2.2. Group actions

A (left) action of a group G on a set X is a map

$$G \times X \to X, \quad (g, x) \mapsto g \cdot x,$$

such that:

1. $e \cdot x = x$ for all $x \in X$.
2. $(g_1 g_2) \cdot x = g_1 \cdot (g_2 \cdot x)$ for all $g_1, g_2 \in G$ and $x \in X$.

An action encodes "symmetries" of objects in X.

2.3. Orbits and stabilizers

For $x \in X$:

- The orbit of x is

$$\mathrm{Orb}(x) = G \cdot x = \{g \cdot x : g \in G\}.$$

- The stabilizer of x is

$$\mathrm{Stab}(x) = G_x = \{g \in G : g \cdot x = x\}.$$

Two elements $x, y \in X$ lie in the same orbit if and only if $y = g \cdot x$ for some $g \in G$. Thus the orbit set X/G is exactly the set of equivalence classes under the relation "reachable by the action."

---

3. Orbit–stabilizer theorem

3.1. Stabilizers are subgroups

Lemma 3.1. For any action of G on X and any $x \in X$, the stabilizer G_x is a subgroup of G.

Proof.

- Identity: $e \cdot x = x$, so $e \in G_x$.
- Closure: if $g, h \in G_x$, then $(gh) \cdot x = g \cdot (h \cdot x) = g \cdot x = x$, so $gh \in G_x$.
- Inverses: if $g \in G_x$, then $g \cdot x = x$. Apply g^{-1} to both sides:

$$g^{-1} \cdot (g \cdot x) = (g^{-1}g) \cdot x = e \cdot x = x,$$

so $g^{-1} \cdot x = x$ and $g^{-1} \in G_x$.

Thus $G_x \leq G$. □

3.2. Bijection between cosets and the orbit

Fix $x \in X$. Define a map

$$\phi : G \to \mathrm{Orb}(x), \quad \phi(g) = g \cdot x.$$

This is surjective by definition of the orbit. The key observation is how fibers of ϕ relate to cosets of G_x.

Lemma 3.2. For $g, h \in G$, $\phi(g) = \phi(h)$ if and only if $h^{-1}g \in G_x$.

Proof.

$\phi(g) = \phi(h)$ means $g \cdot x = h \cdot x$. Apply h^{-1}:

$$h^{-1} \cdot (g \cdot x) = (h^{-1}g) \cdot x = h^{-1} \cdot (h \cdot x) = e \cdot x = x.$$

So $(h^{-1}g) \cdot x = x$, i.e., $h^{-1}g \in G_x$. The reverse direction follows by reversing the steps. □

Lemma 3.2 implies that $\phi(g) = \phi(h)$ exactly when g and h lie in the same left coset of G_x in G. Therefore, the induced map

$$\overline{\phi} : G/G_x \to \mathrm{Orb}(x), \quad gG_x \mapsto g \cdot x$$

is well-defined and bijective.

3.3. Orbit–stabilizer theorem

Theorem 3.3 (Orbit–stabilizer). If G is finite and acts on X, then for every $x \in X$,

$$|\mathrm{Orb}(x)| = \frac{|G|}{|G_x|},$$

equivalently,

$$|G| = |G_x| \cdot |\,\mathrm{Orb}(x)|.$$

Proof.

Since $\overline{\phi}$ is a bijection between $\mathrm{Orb}(x)$ and the set of left cosets G/G_x, we have $|\mathrm{Orb}(x)| = [G : G_x]$. For finite groups, $[G : G_x] = |G|/|G_x|$ by coset counting. □

This theorem is a structural backbone for orbit-counting arguments, and it will also be used implicitly in the proof of Burnside's lemma.

---

4. Burnside's lemma

4.1. Fixed points

For $g \in G$, define the fixed-point set

$$\mathrm{Fix}(g) = X^g = \{x \in X : g \cdot x = x\},$$

and let $|X^g|$ denote its size.

4.2. The lemma

Theorem 4.1 (Burnside / Cauchy–Frobenius). If a finite group G acts on a finite set X, then the number of orbits is

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|.$$

Proof (double counting).

Consider the set of pairs

$$\Omega = \{(g, x) \in G \times X : g \cdot x = x\}.$$

Count $|\Omega|$ in two ways:

1. Fix $g \in G$ first. The number of x with $g \cdot x = x$ is $|X^g|$. Summing over g:

$$|\Omega| = \sum_{g \in G} |X^g|.$$

2. Fix $x \in X$ first. The number of g with $g \cdot x = x$ is $|G_x|$. Summing over x:

$$|\Omega| = \sum_{x \in X} |G_x|.$$

Now regroup the second sum by orbits. For a given orbit $O \subseteq X$, choose any representative $x \in O$. For every $y \in O$, orbit–stabilizer gives $|G_y| = |G_x|$ (stabilizers of orbit points are conjugate and hence have equal size), and $|O| = |G|/|G_x|$. Therefore,

$$\sum_{y \in O} |G_y| = |O| \cdot |G_x| = \frac{|G|}{|G_x|} \cdot |G_x| = |G|.$$

Summing over all orbits:

$$\sum_{x \in X} |G_x| = |G| \cdot |X/G|.$$

Equate the two counts of $|\Omega|$ and divide by $|G|$:

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|.$$

□

Burnside's lemma transforms the orbit-counting task into the computation of $|X^g|$ for each group element g.

---

5. The action of S_n on labeled graphs

5.1. The object set and the group

Let X_n be the set of all labeled simple graphs on vertex set $[n]$. Let $G = S_n$.

Define the action as relabeling vertices:
for $\sigma \in S_n$ and a graph $G = ([n], E)$,

$$\sigma \cdot E = \{\{\sigma(i), \sigma(j)\} : \{i, j\} \in E\}.$$

This action is well-defined, preserves simplicity and undirectedness, and satisfies the group action axioms:

- identity permutation preserves every edge set,
- composition of permutations corresponds to composition of relabelings.

Thus, isomorphism classes of graphs on n vertices are exactly the orbits X_n/S_n.

5.2. Burnside reduction

Burnside yields

$$a(n) = |X_n/S_n| = \frac{1}{n!} \sum_{\sigma \in S_n} |\operatorname{Fix}(\sigma)|.$$

So we must compute, for each permutation σ, how many labeled graphs are fixed by relabeling via σ.

---

6. Fixed graphs under a permutation: edge-orbit structure

6.1. Graphs as 2-colorings of edges

Identify each graph with a function (a 2-coloring)

$$\chi : \binom{[n]}{2} \to \{0,1\},$$

where $\chi(\{i,j\}) = 1$ means the edge is present and $\chi(\{i,j\}) = 0$ means absent.

The permutation σ induces a permutation $\sigma^{(2)}$ on the edge set $\binom{[n]}{2}$ by

$$\sigma^{(2)}(\{i,j\}) = \{\sigma(i), \sigma(j)\}.$$

A coloring χ is fixed by σ if and only if it is constant on each cycle (orbit) of $\sigma^{(2)}$. Therefore, if $\sigma^{(2)}$ decomposes $\binom{[n]}{2}$ into $c(\sigma)$ cycles, then each cycle may be colored arbitrarily with either 0 or 1, giving

$$|\mathrm{Fix}(\sigma)| = 2^{c(\sigma)}.$$

Hence

$$a(n) = \frac{1}{n!} \sum_{\sigma \in S_n} 2^{c(\sigma)}.$$

The problem is now purely group-theoretic/combinatorial: compute $c(\sigma)$ from the cycle structure of σ.

6.2. Cycle decomposition of σ and decomposition of edges

Let $\sigma \in S_n$ have disjoint cycle decomposition with cycle lengths

$$b_1, b_2, \ldots, b_r, \qquad \sum_{t=1}^{r} b_t = n.$$

Let these vertex-cycles be $C_1, \ldots, C_r$ with $|C_t| = b_t$.

Partition edges into two kinds:

1. **Internal edges:** both endpoints lie in the same cycle C_t.
2. **Cross edges:** endpoints lie in different cycles C_s and C_t with $s \neq t$.

We compute the number of $\sigma^{(2)}$-cycles contributed by each kind and add them.

---

7. Counting edge cycles induced by a single vertex-cycle

7.1. Internal edges in one cycle of length b

Consider one vertex-cycle C of length b:

$$C = (v_0\, v_1\, \dots\, v_{b-1}),$$

so $\sigma(v_i) = v_{i+1 \bmod b}$.

An internal edge corresponds to an unordered pair $\{v_i, v_j\}$ with $i \neq j$. Define the cyclic distance

$$d(i,j) = \min\{|i-j|,\, b - |i-j|\} \in \{1, 2, \dots, \lfloor b/2 \rfloor\}.$$

Lemma 7.1. Two internal edges $\{v_i, v_j\}$ and $\{v_{i'}, v_{j'}\}$ lie in the same orbit under $\sigma^{(2)}$ if and only if $d(i,j) = d(i',j')$.

Proof.

Applying $\sigma^{(2)}$ sends $\{v_i, v_j\}$ to $\{v_{i+1}, v_{j+1}\}$. This shifts both indices by $+1$ mod b, so the difference $j - i$ mod b is invariant up to sign, and thus the minimal distance $d(i,j)$ is invariant. Hence edges in the same orbit must share the same d.

Conversely, if $d(i,j) = d(i',j') = d$, then (possibly swapping endpoints) we may assume $j = i + d$ or $j = i - d$ mod b. By applying a suitable power σ^t, we can shift i to i' and obtain $\{v_{i'}, v_{i'+d}\}$, showing all edges of the same distance d lie in a single orbit. $\square$

Therefore, internal edges split into exactly one orbit for each possible distance $d \in \{1, \dots, \lfloor b/2 \rfloor\}$.

Corollary 7.2. A vertex-cycle of length b contributes exactly $\lfloor b/2 \rfloor$ cycles to $c(\sigma)$ from internal edges.

This matches geometric intuition: place vertices on a regular b-gon, then cyclic rotation partitions chords by chord length, and there are $\lfloor b/2 \rfloor$ distinct chord lengths.

---

8. Counting edge cycles between two vertex-cycles

8.1. Cross edges between cycles of lengths a and b

Let C and D be two disjoint vertex-cycles of lengths a and b:

$$C = (u_0\, u_1\, \dots\, u_{a-1}), \qquad D = (w_0\, w_1\, \dots\, w_{b-1}),$$

with $\sigma(u_i) = u_{i+1 \bmod a}$ and $\sigma(w_j) = w_{j+1 \bmod b}$.

A cross edge is $\{u_i, w_j\}$. Under $\sigma^{(2)}$ it maps to $\{u_{i+1}, w_{j+1}\}$. Thus the orbit of a cross edge advances indices synchronously:

$$(u_i, w_j) \mapsto (u_{i+t}, w_{j+t}) \quad (\bmod\, a \text{ and } \bmod\, b).$$

The smallest positive t returning to the original pair is the least common multiple:

$$t = \operatorname{lcm}(a, b).$$

Hence each orbit has length $\operatorname{lcm}(a, b)$.

There are $a \cdot b$ cross edges between C and D, so the number of distinct orbits is

$$\frac{ab}{\operatorname{lcm}(a, b)} = \gcd(a, b).$$

Lemma 8.1. Two vertex-cycles of lengths a and b contribute exactly $\gcd(a, b)$ edge-cycles to $c(\sigma)$ from cross edges between them.

This is a standard synchronization phenomenon: the relative phase between the two cycles takes values in $\mathbb{Z}_{\gcd(a,b)}$.

---

9. The fixed-point exponent $c(\sigma)$

Combine internal and cross contributions.

Let σ have cycle lengths $b_1, \dots, b_r$. Then

- internal contribution: $\sum_{i=1}^{r} \lfloor b_i/2 \rfloor$,
- cross contribution: $\sum_{1 \le i < j \le r} \gcd(b_i, b_j)$.

Therefore:

Theorem 9.1. For $\sigma \in S_n$ with cycle lengths $b_1, \dots, b_r$,

$$c(\sigma) = \sum_{i=1}^{r} \left\lfloor \frac{b_i}{2} \right\rfloor + \sum_{1 \le i < j \le r} \gcd(b_i, b_j).$$

Consequently,

$$|\mathrm{Fix}(\sigma)| = 2^{c(\sigma)}.$$

Plugging this into Burnside gives the conceptual solution:

$$a(n) = \frac{1}{n!} \sum_{\sigma \in S_n} 2^{c(\sigma)}.$$

The remaining issue is computational: summing over all $n!$ permutations is infeasible for moderate n, so we must regroup the sum by cycle type.

---

10. Regrouping Burnside's sum by cycle type

10.1. Cycle type multiplicities

Let a permutation σ have m_ℓ cycles of length ℓ for $\ell = 1, 2, \dots, n$, so

$$\sum_{\ell=1}^{n} \ell \, m_\ell = n.$$

This data $(m_1, \ldots, m_n)$ is equivalent to an integer partition $\lambda \vdash n$ with multiplicities.

Since $c(\sigma)$ depends only on the multiset of cycle lengths, it depends only on λ, so we write $c(\lambda)$.

Using multiplicities, we can rewrite $c(\lambda)$ more explicitly:

- internal edges: each ℓ-cycle contributes $\lfloor \ell/2 \rfloor$, so total internal part is $\sum_\ell m_\ell \lfloor \ell/2 \rfloor$,
- cross edges between an ℓ-cycle and an s-cycle contribute $\gcd(\ell, s)$. The number of such unordered pairs of cycles is:
  - $m_\ell m_s$ if $\ell \neq s$,
  - $\binom{m_\ell}{2}$ if $\ell = s$.
    Thus

$$c(\lambda) = \sum_{\ell \geq 1} m_\ell \left\lfloor \frac{\ell}{2} \right\rfloor + \sum_{1 \leq \ell < s \leq n} m_\ell m_s \, \gcd(\ell, s) + \sum_{\ell \geq 1} \binom{m_\ell}{2} \ell.$$

10.2. Counting permutations of a fixed cycle type

Lemma 10.1. The number of permutations in S_n with cycle multiplicities $(m_1, \ldots, m_n)$ is

$$\frac{n!}{\prod_{\ell=1}^{n} \ell^{m_\ell} m_\ell!}.$$

Proof (explicit construction and overcounting correction).

Start from a list of the n symbols. There are $n!$ ways to arrange them linearly. We want to interpret this arrangement as an ordered list of cycles, where:

- the first ℓ symbols form the first ℓ-cycle,
- the next ℓ symbols form the next ℓ-cycle,
  and so on, according to the required multiset of cycle lengths.

However, two kinds of overcounting occur:

1. **Rotation within each cycle.**

   A cycle $(v_1\, v_2\, \ldots\, v_\ell)$ is the same as $(v_2\, \ldots\, v_\ell\, v_1)$, giving ℓ equivalent linear representations. For each of the m_ℓ cycles of length ℓ, we must divide by ℓ. This yields a factor ℓ^{m_ℓ} in the denominator.

2. **Reordering cycles of equal length.**

   Cycles are disjoint, and the product of disjoint cycles commutes; hence permuting the m_ℓ cycles of the same length ℓ does not change the permutation. Therefore we must divide by $m_\ell!$ for each ℓ.

Combining corrections gives the stated formula. □

10.3. The final partition formula

Regroup Burnside's sum by cycle type:

$$a(n) = \frac{1}{n!} \sum_{\sigma \in S_n} 2^{c(\sigma)} = \frac{1}{n!} \sum_{\lambda \vdash n} \left(\#\{\sigma : \text{type}(\sigma) = \lambda\} \right) 2^{c(\lambda)}.$$

Insert Lemma 10.1:

$$a(n) = \frac{1}{n!} \sum_{\lambda \vdash n} \frac{n!}{\prod_{\ell \geq 1} \ell^{m_\ell} m_\ell!} \, 2^{c(\lambda)} = \sum_{\lambda \vdash n} \frac{2^{c(\lambda)}}{\prod_{\ell \geq 1} \ell^{m_\ell} m_\ell!}.$$

This is a standard closed form for the number of unlabeled graphs on n vertices. The sequence $(a(n))_{n \geq 1}$ is widely tabulated in the literature and in OEIS (A000088).

---

11. Small-n sanity checks (illustrative)

11.1. Case $n = 3$

Integer partitions of 3 are:

- 3 (one 3-cycle): $m_3 = 1$.
- $2 + 1$ (one 2-cycle and one fixed point): $m_2 = 1, m_1 = 1$.
- $1 + 1 + 1$ (identity): $m_1 = 3$.

Compute $c(\lambda)$:

1. $\lambda = 1^3$:

$c = \sum m_\ell \lfloor \ell/2 \rfloor + \sum \binom{m_\ell}{2} \ell + \sum_{\ell<s} m_\ell m_s \gcd(\ell, s)$.

Here $m_1 = 3$:

internal: $3\lfloor 1/2 \rfloor = 0$,

same-length cross: $\binom{3}{2} \cdot 1 = 3$,

so $c = 3$, and weight is $2^3/(1^3 \cdot 3!) = 8/6$.

2. $\lambda = 2 + 1$:

   internal: $\lfloor 2/2 \rfloor + \lfloor 1/2 \rfloor = 1$,

   cross between cycles: $\gcd(2, 1) = 1$,

   same-length cross: none,

   so $c = 2$. Denominator is $2^1 \cdot 1^1 \cdot 1! \cdot 1! = 2$, weight $2^2/2 = 2$.

3. $\lambda = 3$:

   internal: $\lfloor 3/2 \rfloor = 1$,

   cross: none,

   so $c = 1$. Denominator $3^1 \cdot 1! = 3$, weight $2^1/3$.

Sum:

$$a(3) = \frac{8}{6} + 2 + \frac{2}{3} = 4,$$

which matches the four unlabeled graphs on three vertices.

---

12. Algorithmic remarks

The partition formula

$$a(n) = \sum_{\lambda \vdash n} \frac{2^{c(\lambda)}}{\prod_{\ell \geq 1} \ell^{m_\ell} m_\ell!}$$

suggests a practical computation strategy when n is moderate:

1. Enumerate integer partitions $\lambda \vdash n$ (equivalently multiplicity vectors $(m_1, \ldots, m_n)$).
2. For each partition, compute $c(\lambda)$ using the multiplicity-based formula:

$$c(\lambda) = \sum_{\ell \geq 1} m_\ell \left\lfloor \frac{\ell}{2} \right\rfloor + \sum_{1 \leq \ell < s \leq n} m_\ell m_s \, \gcd(\ell, s) + \sum_{\ell \geq 1} \binom{m_\ell}{2} \ell.$$

3. Accumulate the rational weight $2^{c(\lambda)} / \prod_\ell \ell^{m_\ell} m_\ell!$ in the desired modulus by multiplying with modular inverses.

The bottleneck is the number of partitions $p(n)$, which grows subexponentially but rapidly. This method is therefore well-suited for small-to-medium n (as commonly used in template-style enumeration tasks), and it directly reflects the underlying group-theoretic classification by conjugacy classes of S_n.

---

13. Conclusion

We have provided a detailed derivation of the enumeration of unlabeled simple graphs on n vertices using group actions. The core steps are:

1. Model unlabeled graphs as orbit classes of labeled graphs under the action of S_n.
2. Apply Burnside's lemma to express the orbit count as an average of fixed-point counts.
3. Interpret graphs as 2-colorings of edges and show that fixed colorings are determined by edge orbits, giving $|\mathrm{Fix}(\sigma)| = 2^{c(\sigma)}$.
4. Derive $c(\sigma)$ explicitly from the cycle decomposition of σ by analyzing induced action on unordered vertex pairs.
5. Regroup the Burnside sum by cycle type (conjugacy classes) and count permutations of each type to obtain the partition formula.

This approach exemplifies how foundational group-theoretic tools—actions, stabilizers, and orbit counting—convert an isomorphism enumeration problem into an explicit and computable expression.

References

1. van Lint, J. H., & Wilson, R. M. (2001). A Course in Combinatorics (2nd ed.). Cambridge

University Press.

- Publisher page (Cambridge Core): https://www.cambridge.org/core/books/course-in-combinatorics/84B89574496561D5E3A960863B85E4B7
- ISBN/Front Matter PDF (Cambridge): https://assets.cambridge.org/97805210/06019/frontmatter/9780521006019_frontmatter.pdf
- Book info (Google Books preview): https://books.google.com/books/about/A_Course_in_Combinatorics.html?id=5l5ps2JkyT0C
- Full-text mirror (secondary): https://mathematicalolympiads.wordpress.com/wp-content/uploads/2012/08/a_course_in_combinatorics.pdf

2. Cameron, P. J. (1994). Combinatorics: Topics, Techniques, Algorithms. Cambridge University Press.
   - Google Books listing: https://books.google.com/books/about/Combinatorics.html?id=_aJIKWcifDwC
   - Amazon book details: https://www.amazon.com/Combinatorics-Techniques-Algorithms-Peter-Cameron/dp/0521457610
   - Full-text reference PDF (Scribd): https://www.scribd.com/document/442427161/Peter-J-Cameron-Combinatorics-topics-techniques-algorithms-Cambridge-University-Press-1995-pdf
3. Harary, F., & Palmer, E. M. (1973). Graphical Enumeration. Academic Press.
   - WorldCat entry: https://www.worldcat.org/title/graphical-enumeration/oclc/678509
   - Secondary reference summary (OEIS A000088 links): https://oeis.org/A000088/internal
4. Pólya, G., & Read, R. C. (1987). Combinatorial Enumeration of Groups, Graphs, and Chemical Compounds. Springer.
   - Springer book info: https://link.springer.com/book/10.1007/978-1-4612-4930-1
   - Google Books preview: https://books.google.com/books/about/Combinatorial_Enumeration_of_Groups_Graphs.html?id=ZhVtQgAACAAJ
   - Secondary reference checklist (OEIS A000088 citations): https://oeis.org/A000088/internal
5. Stanley, R. P. (2011). Enumerative Combinatorics, Volume 1 (2nd ed.). Cambridge University Press.
   - Publisher page (Cambridge Core): https://www.cambridge.org/core/books/enumerative-combinatorics/AA7C6552765C252AD5F7D49F0752EFC3

- Google Books preview: https://books.google.com/books/about/Enumerative_Combinatorics_Volume_1.html?id=hK0TEAAAQBAJ
- Secondary reference (OEIS context): https://oeis.org/A000088/internal

6. Isaacs, I. M. (2008). Finite Group Theory. American Mathematical Society.
   - AMS book page: https://bookstore.ams.org/gsm-92/
   - Google Books listing: https://books.google.com/books/about/Finite_Group_Theory.html?id=24XyBwAAQBAJ
7. OEIS Foundation Inc. (n.d.). The On-Line Encyclopedia of Integer Sequences: A000088 — Number of unlabeled simple graphs on n nodes. Retrieved January 10, 2026, from https://oeis.org/A000088.
   - Main entry: https://oeis.org/A000088
   - Internal-format view: https://oeis.org/A000088/internal
   - Sequence data file (B-file): https://oeis.org/A000088/b000088.txt
8. Read, R. C. (1959). The enumeration of locally restricted graphs (I). Journal of the London Mathematical Society, s1-34(4), 417–436.
   - DeepDyve entry: https://www.deepdyve.com/doc-view/10.1112/jlms/s1-34.4.417
   - Semantic Scholar summary: https://www.semanticscholar.org/paper/The-Enumeration-of-Locally-Restricted-Graphs-%28I%29-Read/cf51848cb57ae37165624634af53cc15bfe1b04e
9. Wright, E. M. (1972). The number of unlabeled graphs with many nodes and edges. Bulletin of the American Mathematical Society, 78(6), 1032–1034.
   - PDF reference (SciSpace): https://scispace.com/pdf/the-number-of-unlabelled-graphs-with-many-nodes-and-edges-3pp90h00rj.pdf
   - Bulletin AMS abstract: https://academic.oup.com/bulletin/article/78/6/1032/1657780
10. Introduction to Group Theory (my own blog)

    https://www.luogu.com.cn/article/dr4dv54v

    http://blog.tsawke.com/?title=GroupTheory

Appendix. C++ Implementation

```cpp
#define _USE_MATH_DEFINES
#include <bits/stdc++.h>

#define PI M_PI
#define E M_E
#define npt nullptr
#define SON i->to
#define OPNEW void* operator new(size_t)
#define ROPNEW(arr) void* Edge::operator new(size_t){static Edge* P = arr;
return P++;}

using namespace std;

mt19937 rnd(random_device{}());
int rndd(int l, int r){return rnd() % (r - l + 1) + l;}
bool rnddd(int x){return rndd(1, 100) <= x;}

typedef unsigned int uint;
typedef unsigned long long unll;
typedef long long ll;
typedef long double ld;

#define MOD (997)

template < typename T = int >
inline T read(void);

int N;
ll fact[110], inv[110], inv_d[110];
int cnt[110];
ll ans(0);
basic_string < int > cur;
unordered_set < int > exist;

ll qpow(ll a, ll b){
    ll ret(1), mul(a);
    while(b){
```

```
        if(b & 1)ret = ret * mul % MOD;
        b >>= 1;
        mul = mul * mul % MOD;
    }return ret;
}

void Init(void){
    for(int i = 1; i <= 100; ++i)inv_d[i] = qpow(i, MOD - 2);
    fact[0] = 1;
    for(int i = 1; i <= 100; ++i)fact[i] = fact[i - 1] * i % MOD;
    inv[100] = qpow(fact[100], MOD - 2);
    for(int i = 99; i >= 0; --i)inv[i] = inv[i + 1] * (i + 1) % MOD;
}

void dfs(int lft = N){
    if(!lft){
        ll C(0);
        for(auto i : cur)C += i >> 1;
        for(int i = 1; i <= (int)cur.size(); ++i)
            for(int j = 1; j <= i - 1; ++j)
                C += __gcd(cur.at(i - 1), cur.at(j - 1));
        ll ret = qpow(2, C);
        for(auto i : cur)(ret *= inv_d[i]) %= MOD;
        for(auto i : exist)(ret *= inv[cnt[i]]) %= MOD;
        (ans += ret) %= MOD;
        return;
    }
    for(int i = cur.empty() ? 1 : cur.back(); i <= lft; ++i){
        cur += i, ++cnt[i], exist.insert(i);
        dfs(lft - i);
        cur.pop_back();
        if(!--cnt[i])exist.erase(i);
    }
}

int main(){
    Init();
    N = read();
    dfs();
    printf("%lld\n", ans);
```

```
    fprintf(stderr, "Time: %.6lf\n", (double)clock() / CLOCKS_PER_SEC);
    return 0;
}

template < typename T >
inline T read(void){
    T ret(0);
    int flag(1);
    char c = getchar();
    while(c != '-' && !isdigit(c))c = getchar();
    if(c == '-')flag = -1, c = getchar();
    while(isdigit(c)){
        ret *= 10;
        ret += int(c - '0');
        c = getchar();
    }
    ret *= flag;
    return ret;
}
```